



Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Fran Babić, Toni Brajko, Ivan Janjić, Martina Licul, Patrick Pavić, and Petar Sruk.

Implementation examples are given in attached source code files.

Task Eurokod

Prepared by: Martina Licul

Necessary skills: arrays, sorting

To solve this task we will create two arrays, one in which we will store the total number of points of the i -th competitor, and the other in which we will store the number of points that the members of the association have assigned to the i -th competitor.

Let's look at the president's scoring. If the i -th place of his ranking is marked with a_i , then the a_i -th competitor gets $n - i + 1$ points, where n is the total number of competitors.

Let's look at the scoring of the members of the association. In the first subtask, it is true that for each code the number of votes of the members for that code is equal to the number of points they assigned to it. In that case, we add the number of votes of the members for his code to the total number of points of the i -th competitor. Let's move on to the general case, when it is not true that the votes and points of the members of the association match. Let's create an array of pairs (b_i, i) , where b_i is the number of votes of the members of the association for the i -th code. Sort that array in ascending order by the number of votes. Now we will assign points to the competitors using that array. Let the pair (b_i, i) be at the j -th place. Then the i -th code is assigned j points by the members of the association.

For example, let the votes of the members of the association be 50 10 20, the array of pairs before sorting is $(50, 1)$ $(10, 2)$ $(20, 3)$, and after sorting $(10, 2)$ $(20, 3)$ $(50, 1)$. Then the second code is assigned 1 point, the third 2 points, and the first 3 points.

Let's now create an array where each element consists of three numbers, the first number is the total number of points of the i -th competitor, the second number is the number of points that the i -th competitor received from the members of the association, and the third number is the code label (i.e. the number i).

Sort that array in descending order by the first number, and if the first numbers are equal, then in descending order by the second number.

The sorted array is also the final ranking of competitors. It only remains to print the ranking of competitors in the appropriate format.

Task Vrsar

Prepared by: Vito Anić

Necessary skills: greedy algorithms

The most important observation for this task is that it will always be possible to have an optimal solution that uses only one ice rink.

Proof: Let us assume that some optimal solution uses multiple ice rinks. Look at the ice rink where Iva and Mia skate last. The solution would be the same if they went directly to that ice rink. They will not spend any more time traveling to it (because they have to reach it in both solutions), and every minute that was spent skating on other ice rinks could be spent skating on the last one.

With that observation, we can simplify the task a bit. We do not need the time to descend from hills. For every starting position, we have to find the maximum of $t_i - \text{abs}(a - x_i)$ for every i . That can be done in many ways; the official solution uses two sets, one for the positions left of the current, and one for the



positions right of the current position. For details, look at the attached implementation.

Task Milano C.le

Prepared by: Vito Anić and Martina Licul

Necessary skills: longest increasing subsequence

We will use the notation from the task text: (a_i) is the order in which the trains arrived at the station on the first day, and (b_i) is the order in which the trains left the station on the second day.

Let's create a sequence (c_i) , such that c_i is equal to b_j for which $a_j = i$. In other words, sort the order of departure of trains according to the order of arrival of trains.

For example, let the order of arrival of trains be 3 5 2 4 1, and the order of departure 3 2 5 1 4. Then $c_1 = b_5 = 4$ because $a_5 = 1$, $c_2 = b_3 = 5$ because $a_3 = 2$, $c_3 = b_1 = 3$ because $a_1 = 3$, $c_4 = b_4 = 1$ because $a_4 = 4$, and $c_5 = b_2 = 2$ because $a_2 = 5$. Therefore, the sequence $(c_i) = 4 5 3 1 2$.

Let's now look at the trains in the sequence (c_i) . In the example above, trains c_1 and c_2 cannot be on the same platform because $c_1 < c_2$, which means that train c_1 would have to leave the station before train c_2 , which is impossible because train c_1 arrived at the station before train c_2 . On the other hand, trains c_1 and c_3 can be on the same platform because $c_1 > c_3$, which means that train c_3 will want to leave the station before train c_1 , which is possible because train c_3 arrived at the station after train c_1 . From this it follows that if we were to find a sequence such that $c_{x_1} < c_{x_2} < \dots < c_{x_k}$, then we would need k platforms for those trains.

We have now reduced the task to finding the longest such sequence. This problem is known as the *longest increasing subsequence*. The solution to the task is the length of the longest increasing subsequence of the sequence (c_i) . Note that the trains contained in that subsequence will be the first train on each of the platforms, and that it is possible to arrange the other trains on the platforms so that all trains can leave the station without being blocked by other trains. The overall time complexity is $\mathcal{O}(n \log n)$.

Task Restorani

Prepared by: Toni Brajko and Ivan Janjić

Necessary skills: dfs, constructive algorithms

We can reformulate the task in the following way: given a tree of size n and two sets of nodes A and B of size m each, we need to find the length and construct the shortest path that begins and ends in node 1, alternates between nodes from set A and nodes from set B , and visits each node from both sets at least once.

Let's call a node x *interesting* if $x \in A$, $x \in B$ or $x = 1$. Notice that there are $\mathcal{O}(m)$ interesting nodes.

Let $\text{dp}[i][X][Y]$ be minimal length of a path if we are currently in node i , we've already visited all nodes from set $X \subseteq A$ and set $Y \subseteq B$. If $|X| = |Y|$, node i that we are currently in belongs to set Y . The transition is:

$$\text{dp}[i][X][Y] = \min(\text{dp}[j][X][Y \setminus \{i\}] + d[j][i], j \in X),$$

where $d[i][j]$ is the distance between interesting nodes i and j . Otherwise, $|X| = |Y| + 1$ and node i belongs to set X . Transition is similar as the one mentioned. We can solve this with dynamic programming with bitmasks. The answer is then contained in $\text{dp}[1][A][B]$, and we can construct the solution by backtracking our dp.

To calculate the minimum path length, let's consider each edge separately and determine the minimum number of times it must be traversed. This edge divides the tree into two subtrees. Let a be the number of nodes from set A in one of these subtrees, and b be the number of nodes from set B in the same subtree. If $a = b = 0$, this edge will never need to be traversed, and its contribution to the answer is 0. Otherwise, we



must traverse this edge at least $2 \max(1, |a - b|)$ times. We claim that there always exists a construction that traverses each edge exactly this many times!

In the second subtask, the tree is a chain, and we can identify it with a sequence of length n . For an arbitrary sequence, let $f_A(i)$ be the number of elements from set A in the prefix of the sequence of length i . Similarly, let $f_B(i)$ be the number of elements from set B in the prefix of the sequence of length i . Let's call a sequence of length k *positive* if it satisfies $f_A(k) = f_B(k)$ and $f_A(i) \geq f_B(i)$ for all $i \leq k$. Analogously, let's call a sequence of length k *negative* if it satisfies $f_A(k) = f_B(k)$ and $f_A(i) \leq f_B(i)$ for all $i \leq k$. The entire sequence can now be divided into consecutive positive and negative subsequences (you will best understand this with an example).

Observation. *Any negative sequence in reverse order is positive.*

We can traverse a positive sequence by pairing the i -th smallest element from set A with the i -th smallest element from set B , from the beginning of the sequence.

First, we will traverse the entire sequence from left to right (from node 1 to the last interesting node), visiting all positive subsequences. On the way back to node 1, we will traverse all negative subsequences (which, due to the opposite direction, will effectively be positive). It is easy to verify that this achieves the minimum number of passes through each edge. The complexity of this solution is $O(n + m)$.

For an arbitrary tree, similar to the second subtask, let $f_A(i)$ be the number of elements from set A in the subtree rooted at node i (including the node itself), and let $f_B(i)$ be the number of elements from set B in the subtree rooted at node i . Let's call the subtree rooted at node i *positive* if $f_A(i) \geq f_B(i)$, and *negative* if $f_A(i) \leq f_B(i)$. Note that a subtree without interesting nodes is both positive and negative. We will never need to traverse such subtrees, so this case is ignored in the following text.

We solve the third subtask by traversing the tree node by node. We say we are *in transition* if the last interesting node we visited belonged to set A (and now we are on the way to a node from set B). Otherwise, we say we are *not in transition*.

Let's assume we are currently at node i . We propose the following naive algorithm:

- If we are not in transition and $i \in A$, add i to the solution path, and $A := A \setminus \{i\}$.
- If we are in transition and $i \in B$, add i to the solution path, and $B := B \setminus \{i\}$.
- If we are not in transition and there is a child j of node i whose subtree is positive, move to j .
- If we are in transition and there is a child j of node i whose subtree is negative, move to j .
- If none of these conditions is satisfied, go to the parent of node i (or terminate the algorithm if we are at node 1).

There are many incorrect algorithms, but the key idea is to ensure that if a subtree has elements from both sets, we will pair them within that subtree (without unnecessary exiting and returning to the subtree). This ensures that we traverse each edge the minimum required number of times.

Each time we change the sets A or B , we need to recalculate $f_A(i)$ and $f_B(i)$ for each i . This can be done in $O(n)$, and we will change the sets $O(m)$ times. Therefore, the overall complexity of this algorithm is $O(n \cdot m)$.

In the last subtask, we cannot go node by node; instead, we need a better algorithm to determine the order of visiting interesting locations. For each node, we will maintain the optimal order of visiting interesting locations in its subtree and sets A and B of locations that need to be paired with some locations outside the subtree of that node.

Observation. *We will never have $|A| > 0$ and $|B| > 0$ for any node.*



Otherwise, with the same intuition as in the previous subtask, we would want to pair elements from different sets within the same subtree (to avoid the need for an additional traversal of the edge leading outside the subtree). Specifically, for a positive subtree, we will have $|A| \geq 0$, and for a negative one, $|B| \geq 0$.

First, we will calculate the required values for each child of node i , and then we will merge the traversals of the children into one traversal. Let's assume that each child has an empty traversal. We will then traverse from positive to negative subtree and vice versa, alternately visiting nodes from set A and set B (until one of these sets becomes empty). If some child j has a non-empty traversal, and for it, $|A| = |B| = 0$, it is sufficient to add the traversal of its subtree to the beginning or end of the overall traversal from node i .

The remaining case is when a child, for which $|A| > 0$ or $|B| > 0$ holds, has a non-empty traversal. Now we need to choose the moment when we will traverse its subtree. However, instead of choosing this when merging multiple subtrees into a larger one, during the alternating traversal of subtrees, we can "attach" the obtained path to some unvisited node. In the future, when passing through that node, we will add the attached traversal and continue the algorithm. The added traversal will leave us in the same subtree, so the overall path will not be unnecessarily extended. This way, we bypass several cases in which our algorithm repeats an edge more times than necessary.

There are multiple ways to write the solution, and to avoid a lengthy implementation, you can find a solution using a stack and a linked list in the provided code. The overall complexity of the algorithm is $O(n)$.

Task Slučajna cesta

Prepared by: Petar Sruk

Necessary skills: probability, dynamic programming, rerooting

First, we'll calculate expected beauty of a path that starts in the root. We can calculate it with a standard dynamic $\text{down}[v] = \text{val}[v] + \sum_{c \in \text{child}[v]} \text{prob}[v][c] \cdot \text{down}[c]$, where $\text{val}[v]$ is the beauty of park v and $\text{prob}[v][c]$ is the probability that Vito goes from park v to park u (from now on we're assuming he starts in the root). It only remains to calculate $\text{prob}[v][c]$.

In third subtask the graph is a binary tree so we know that every vertex can have 0, 1 or 2 children. If v has one child, the probability that the edge is directed downwards is 50% and in that case Vito takes that road. If it has two children we have four cases: both edges directed downwards, only left edge downwards, only right edge downwards and neither edge directed downwards. Each case has a probability of 25%, in second (third) case Vito continues his path towards left (right) child, in fourth case his journey ends, and in the first case he randomly chooses towards which child he'll go with probabilities 50/50. Summing up probabilities that Vito goes towards left child over all cases we get 37.5%. We get the same value for probability he goes towards the right child and we can now solve 50% of third subtask.

Observation. Probability that from vertex v Vito goes to his child c is equal for every child of v .

It follows from the symmetry, and we can convince ourselves by analysing cases for arbitrary pair of children similar to the former analysis.

Since the probability that Vito ends his path in vertex v with k children is 2^{-k} (all edges directed upwards), it follows that probability that he takes a path from v to its child c is equal to $\text{prob}[v][c] = \text{pr}(k) = \frac{1-2^{-k}}{k}$.

Applying dynamic programming to all starting vertices we get a solution with time complexity $O(n^2)$ for complete second subtask.

For final solution we have to apply rerooting. For that we need expected value of a path given that from the starting vertex we go upwards. In case where parent of current vertex is not the root we can calculate that value by adding expected beauty of path upwards of parent and subtracting expected beauty of path starting at parent and going to current vertex: $\text{up}[v] = \text{val}[v] + \text{down}[p] + \text{up}[p] - \text{prob}[p][v] \cdot \text{down}[v]$. And if the parent



is the root then we subtract the expected beauty of path towards current vertex and normalize probability due to the change of number of children: $up[v] = val[v] + \frac{pr(k-1)}{pr(k)} (down[p] - prob[p][v] \cdot (down[v] + val[p]))$, for k the number of children of root $p = 1$.

Now we can calculate the expected beauty of path starting at arbitrary vertex with $down[v]$ and $up[v]$ in $O(1)$, for details refer to implementation. Total time complexity is $O(n)$.

For complete solution of the third subtask it was enough to calculate the probability for $k \leq 3$ and apply rerooting.

Alternatively, we can apply general rerooting algorithm in time complexity $O(n \log n)$.